

Отчёт по лабораторной работе №3

Математические основы защиты информации и информационной безопасности

Ли Хан

Содержание

1	Цель работы	5
2	Анализ выполненного упражнения	6
2.1	Анализ принципа реализации алгоритма	6
2.2	принципа реализации алгоритма компилятора	7
2.3	результат	8
3	Вывод	9

Список иллюстраций

2.1	принципа реализации алгоритма компилятора	7
2.2	принципа реализации алгоритма результат	8

Список таблиц

1 Цель работы

Изучить математическую логику шифрования гаммированием с использованием базы индексации $= 1$. Через детальный пошаговый вывод освоить процесс объединения индексов открытого текста и гаммы при операциях по модулю 33, а также проверить точность реализации алгоритма.

2 Анализ выполненного упражнения

2.1 Анализ принципа реализации алгоритма

Шифрование гаммированием — это метод симметричного шифрования, суть которого заключается в наложении последовательности «гаммы» на символы открытого текста.

- Процесс оцифровки: Каждой букве алфавита присваивается порядковый номер (например, А=1, Б=2 ... Я=33).
- Логика шифрования: Порядковый номер символа текста p_i складывается с номером символа гаммы k_i . Результат вычисляется по модулю N (размер алфавита): $c_i = (p_i + k_i) \pmod{N}$.
- Логика дешифрования: Благодаря свойствам модульной арифметики, обратный процесс выполняется по формуле: $p_i = (c_i - k_i + N) \pmod{N}$.

2.2 принципа реализации алгоритма компилятора

```
def gamma_cipher(text, key, mode='encrypt'):
    # 中文: 定义俄语字母表 (33个字母)
    # RU: Определяем русский алфавит (33 буквы)
    alphabet = "АБВГДЕЁЖЗИЙКЛМНОПРСТУФХЦЧШЩЪ"
    N = len(alphabet)

    text = text.upper()
    key = key.upper()
    result = ""

    # 中文: 遍历明文中的每一个字符
    # RU: Проходим через каждый символ ис:
    for i in range(len(text)):
        if text[i] in alphabet:
            # 中文: 获取当前字符和密钥字符在字母表中的索引位置
            # RU: Получаем индексы текущего
            p_idx = alphabet.index(text[i])
            # 中文: 使用取模运算让密钥(Gamma)循环使用
            # RU: Используем операцию остатка
            k_idx = alphabet.index(key[i % len(key)])

            if mode == 'encrypt':
                # 中文: 加密公式: (明文索引 + 密钥索引) 模 N
                # RU: Формула шифрования: (
                res_idx = (p_idx + k_idx) % N
            else:
                # 中文: 解密公式: (密文索引 - 密钥索引 + N)
                # RU: Формула дешифрования
                res_idx = (p_idx - k_idx + N) % N

            result += alphabet[res_idx]
        else:
            # 中文: 如果字符不在字母表中 (如空格), 原样保留
            # RU: Если символа нет в алфав
            result += text[i]

    return result
```

Рис. 2.1: принципа реализации алгоритма компилятора

2.3 результат

```
# 示例运行 / Пример запуска
plaintext = "ПРИКАЗ"
gamma = "ГАММА"
encrypted = gamma_cipher(plaintext, gamma, mode='encrypt')
print(f"加密结果: {encrypted}")
```

加密结果: ТРХЧАК

Рис. 2.2: принципа реализации алгоритма результат

3 Вывод

В ходе выполнения работы была успешно реализована система шифрования с использованием конечной гаммы. Эксперимент подтвердил, что стойкость данного метода напрямую зависит от длины и характеристик гаммы. При наличии ключа симметричность модульных операций позволяет полностью восстановить исходное сообщение без потерь.